

Free AI Graphical Story Automation

Developer Handover Requirements

1. Objective

Build a local automation that converts a short story idea into a **30–60 second vertical graphical story video**.

The automation must:

- Run on the user's own computer
- Use only free and open-source software
- Require no subscriptions
- Use no paid APIs
- Have no per-video generation cost
- Produce videos suitable for Instagram Reels, TikTok and YouTube Shorts

Internet may be required initially to download models and software. After installation, the main workflow should work locally.

2. Required Workflow

The user provides:

- A story idea
- Story type, such as emotional, motivational, mystery or moral
- Preferred visual style
- Desired duration
- Optional character description

The automation then performs these steps:

1. Generates a short story script
2. Divides the script into 6–10 scenes
3. Creates one graphical image for each scene
4. Generates voice narration
5. Creates synchronized captions
6. Adds simple image movement and transitions
7. Adds optional background music

8. Exports a vertical MP4 video
-

3. Required Free Tools

Script and Scene Generation

Use:

- **Ollama**
- A suitable locally running open-source language model

The local model should generate structured JSON containing:

- Title
- Hook
- Narration
- Character description
- Scene descriptions
- Image prompts
- Scene duration
- Closing line

No OpenAI, Claude, Gemini or other paid API should be required.

Image Generation

Preferred option:

- **Foocus with Stable Diffusion XL**

Foocus is free, open source and can operate offline. Its documentation states that an Nvidia GPU with approximately 4 GB VRAM is the minimum supported configuration.

Alternative for more advanced control:

- **ComfyUI**
- Stable Diffusion XL or another locally supported model

The developer should choose the model according to the available computer hardware. Large FLUX models may require considerably more GPU memory, so they should not be the default for ordinary computers.

Image requirements:

- 9:16 vertical composition
- Same visual style across scenes
- Reusable character description in every prompt
- Fixed seed where useful
- Ability to regenerate only one failed scene
- Images saved locally

Voice Generation

Use:

- **Piper TTS**

Piper is a free, local neural text-to-speech engine that offers command-line and Python interfaces.

Requirements:

- At least one English narration voice
- Adjustable speaking speed
- WAV audio output
- Separate narration file for each scene or one combined narration file

No ElevenLabs or paid voice API should be used.

Captions

Captions should primarily be generated directly from:

- Scene narration text
- Duration of each generated audio file

The developer should create an SRT subtitle file programmatically.

Optional fallback:

- Whisper or Whisper.cpp for automatic subtitle alignment

Whisper-based tools can run locally and produce timed subtitle files.

Because the narration script is already available, Whisper should not be required unless better synchronization is needed.

Video Creation

Use:

- **FFmpeg**

FFmpeg should:

- Convert scene images into video segments
- Apply slow zoom and pan effects
- Add fade or crossfade transitions
- Combine narration files
- Add captions
- Add background music
- Export the finished MP4

Existing open-source projects demonstrate that images, narration, subtitles and vertical-video assembly can be handled locally.

Final output:

- Resolution: 1080 × 1920
- Aspect ratio: 9:16
- Format: MP4
- Video codec: H.264
- Audio codec: AAC
- Frame rate: 30 FPS
- Duration: approximately 30–60 seconds

4. Automation Technology

Preferred implementation:

- Python
- Simple local web interface using Gradio or Streamlit
- FFmpeg commands run from Python
- Local folders for project storage

Using Python is preferable to a complicated automation platform because the workflow involves local model execution, audio processing and FFmpeg rendering.

The developer may use self-hosted n8n only as an optional orchestration layer. The complete workflow must still run without paying for n8n Cloud.

5. User Interface

Create a simple local interface containing:

- Story idea field
- Story category dropdown
- Duration selector
- Visual style selector
- Voice selector
- Generate button
- Progress display
- Scene preview
- Final video preview
- Open output folder button

The user should not need to write commands manually after installation.

6. Character Consistency

The automation should create a master character description such as:

A 12-year-old boy with short black hair, a worn blue jacket, brown trousers, large expressive eyes and a slim build.

This exact description should be inserted into every relevant scene prompt.

The system should also use:

- The same image model
- The same style prompt
- The same negative prompt
- Fixed or related seeds
- Optional reference-image support
- Optional Stable Diffusion IP-Adapter or character LoRA support

Perfect character consistency is not guaranteed with free local image models, but the developer should maximize it through consistent prompts and reusable references.

7. Background Music

The system should allow the user to place royalty-free music files in a local folder.

Example folder:

assets/music/

The automation should:

- Select a music file
- Lower its volume beneath narration
- Loop or trim it to video length
- Add fade-in and fade-out
- Allow video generation without music

The developer does not need to integrate a paid music service.

Only music that the user owns or that is licensed for reuse should be included.

8. Project Folder Structure

Each story should create its own project directory:

```
projects/  
  story-name/  
    project.json  
    script.txt
```

```
scenes/  
  scene-01.png  
  scene-02.png  
audio/  
  scene-01.wav  
  scene-02.wav  
subtitles/  
  captions.srt  
output/  
  final-video.mp4
```

This will make it easy to edit, regenerate or recover individual files.

9. Error Handling

The automation should:

- Save progress after every stage
 - Retry failed local-model operations
 - Show clear error messages
 - Avoid restarting the entire story
 - Allow regeneration of one scene
 - Continue from the last completed step
 - Verify that FFmpeg created a valid output
 - Record basic logs in a local file
-

10. Hardware Detection

Before generation, the application should identify:

- Operating system
- Available RAM
- GPU model
- GPU VRAM
- Available disk space

The application should choose an appropriate mode:

GPU Mode

Use local Stable Diffusion image generation when a supported GPU is available.

Low-Spec Mode

Use:

- A lighter or quantized image model
- Lower initial image resolution
- CPU-compatible local language model
- Piper TTS
- FFmpeg

CPU-only image generation may be very slow. The developer should test the workflow against the user's actual computer before finalizing the model.

11. Deliverables

The developer must provide:

- Complete source code
- Windows installation script
- Python requirements file
- FFmpeg setup
- Ollama model setup
- Fooocus or ComfyUI setup
- Piper TTS setup
- Local user interface
- Sample story project
- One successfully generated test video
- README installation guide
- Troubleshooting guide
- Start application shortcut or batch file

The preferred startup process should be:

Double-click start.bat

The browser should then open the local application automatically.

12. Acceptance Criteria

The project is complete when the user can:

1. Enter one story idea.
 2. Click one Generate button.
 3. Receive a short script and scene breakdown.
 4. Generate 6–10 local AI illustrations.
 5. Generate local narration.
 6. Generate synchronized captions.
 7. Combine everything into a 9:16 video.
 8. Download or access the final MP4.
 9. Regenerate an individual scene.
 10. Complete the process without entering payment information or using a paid API.
-

13. Strict Cost Requirement

The developer must not use:

- OpenAI API
- ElevenLabs API
- Midjourney
- Runway
- Paid image-generation APIs
- Paid text-to-speech APIs
- Paid cloud rendering
- Paid automation subscriptions
- Paid storage
- Any tool requiring credits for continued operation

The application must use local processing and free/open-source components.

The only practical costs are the user's existing computer, electricity and internet used to download the software and models. There must be no recurring software or API charge.